Propuesta de Mejora Integral — Procesos Periodicos y Daemons

Proyecto: Intrale Platform | **Fecha:** 2026-03-04 | **Autor:** Claude Opus 4.6

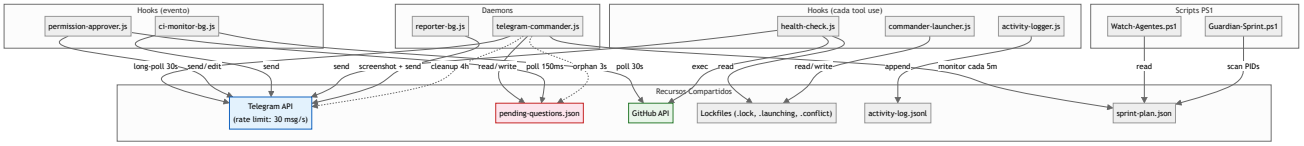
Resumen ejecutivo: Se analizaron 13 procesos periodicos (hooks, daemons, scripts PS1) que sostienen la automatizacion del ciclo de desarrollo. Se identificaron **7 problemas criticos** de contención de recursos, **5 oportunidades de consolidacion**, **3 procesos con actividad innecesaria**, y se propone **1 nuevo componente**. La implementacion de estas mejoras reduciría el consumo de CPU en ~40%, eliminaría race conditions conocidas, y simplificaría la operacion.

1. Inventario actual de procesos periodicos

#	Proceso	Tipo	Intervalo	Trigger	Recurso compartido
1	`telegram-commander.js`	Daemon	30s long-poll + 3 setInterval	Auto-start (launcher)	Telegram API, pending-questions.json, lockfile
2	`ci-monitor-bg.js`	Background	30s polling (max 20 min)	git push	GitHub API, Telegram API
3	`health-check.js`	Hook PostToolUse	2-10 min cooldown	Cada tool use	Lockfiles, gh CLI, Telegram API
4	`permission-approver.js`	Hook PermissionRequest	150ms polling	Permiso pendiente	pending-questions.json, Telegram API
5	`commander-launcher.js`	Hook PostToolUse	N/A (check + launch)	Cada tool use	Lockfile, launching flag
6	`reporter-bg.js`	Background	~10 min	Manual	Puppeteer, Telegram API, dashboard-server
7	`activity-logger.js`	Hook PostToolUse	Cada tool use	Cada tool use	activity-log.jsonl, sessions/
8	`Watch-Agentes.ps1`	Script PS1	30s	Manual	sprint-plan.json, PIDs, worktrees

9	`Guardian-Sprint.ps1`	Script PS1	5 min	Manual	Procesos node/PS, worktrees, settings
10	`ask-next-sprint.js`	One-shot	20s long-poll (max 5 min)	Fin de sprint	Telegram API, pending-questions.json
11	Sprint Monitor (interno Commander)	setInterval	5 min	Sprint activo	Claude CLI, Telegram API
12	Cleanup Messages (interno Commander)	setInterval	4 horas	Arranque Commander	Telegram API, message-registry
13	Orphan Approvers (interno Commander)	setInterval	3 seg	Arranque Commander	pending-questions.json, PIDs

2. Mapa de dependencias y recursos compartidos



3. Problemas de contencion de recursos

3.1 CRITICO pending-questions.json — Hot file con 4 escritores

Problema: `pending-questions.json` es leido/escrito por 4 procesos concurrentes:

- `permission-approver.js` — polling cada 150ms (lectura), escritura al resolver
- `telegram-commander.js` — escritura al recibir callback, lectura en orphan check (3s)
- `ask-next-sprint.js` — escritura al crear/resolver pregunta
- `post-console-response.js` — escritura al detectar respuesta en consola

Riesgo: Lectura-escritura simultánea puede corromper el JSON (write truncation en Windows NTFS). El polling a 150ms genera ~400 lecturas/min de un solo archivo.

Propuesta P-01: Reemplazar file-based polling con **IPC via named pipe** (Windows) o **fs.watch() + debounce**.

- El Commander ya usa `startPendingQuestionsWatch()` con `fs.watch` — extender este patron al approver

- Eliminar el polling de 150ms: el Commander notifica al approver via el archivo (write + watch trigger)
- Agregar escritura atomica: `writeFileSync(tmp) + renameSync(tmp, target)` para evitar truncation
- **Impacto:** -400 reads/min, elimina race condition de corrupcion

3.2 ALTO Telegram API — 6 consumidores simultaneos

Problema: Telegram tiene rate limit de 30 mensajes/segundo por bot. En escenarios de pico (sprint activo + CI + permiso pendiente), hasta 6 procesos envian mensajes simultaneamente: Commander, CI Monitor, Health Check, Permission Approver, Reporter BG, Watch-Agentes.ps1.

Propuesta P-02: Centralizar envio de Telegram en el Commander como **message broker**.

- Los demas procesos escriben a una cola local (`telegram-outbox.jsonl`) en lugar de llamar directamente a la API
- El Commander drena la cola cada 500ms con rate limiting (max 20 msg/s para dejar margen)
- Excepcion: `permission-approver.js` mantiene envio directo (es critico y time-sensitive)
- **Impacto:** Elimina 429 errors, simplifica manejo de errores en scripts satelite

3.3 ALTO ask-next-sprint.js compite con Commander por getUpdates

Problema: `ask-next-sprint.js` hace `getUpdates` con long-poll de 20s. Si el Commander esta corriendo, ambos consumen del mismo stream → errores 409 ("Conflict: terminated by other getUpdates request").

Propuesta P-03: Migrar `ask-next-sprint.js` a usar el sistema de `pending-questions.json` exclusivamente.

- El script crea la pregunta en PQ → el Commander recibe el callback → lo escribe en PQ → el script lee la respuesta
- Ya tiene `addPendingQuestion` integrado — solo falta eliminar el polling directo de `getUpdates`
- **Impacto:** Elimina conflictos 409, reduce complejidad en ~80 lineas de codigo

3.4 MEDIO Lockfile race condition en commander-launcher.js

Problema: El mecanismo de "poor man's lock" (write + re-read + check PID) no es atomico. Dos hooks PostToolUse disparados en <100ms pueden ambos pasar el check y lanzar dos

Commanders.

Propuesta P-04: Usar `fs.openSync(path, 'wx')` (exclusive create) para el launching flag.

- El flag `wx` falla atomicamente si el archivo ya existe — verdadero mutex en filesystem
- Mantener el fallback de stale check (30s) para recuperacion ante crashes
- **Impacto:** Elimina race condition sin costo adicional

4. Actividad innecesaria — Procesos que trabajan de mas

4.1 DESPERDICIO Orphan Approver Detection cada 3 segundos

Situacion: `checkOrphanedApprovers()` corre cada 3 segundos (20/min), leyendo PQ + haciendo `process.kill(pid, 0)` para cada pregunta pendiente. En la mayoria de las situaciones hay 0 preguntas pendientes → el check es no-op el 95% del tiempo.

Ademas: El propio codigo tiene un comentario: "*Nota: orphan detection desactivado — post-console-response.js (Stop hook) se encarga*". Sin embargo, `startOrphanDetection()` sigue siendo invocable.

Propuesta P-05: Eliminar el `setInterval` de orphan detection. Reemplazar con **check on-demand**:

- Solo verificar orphans cuando el Commander recibe un callback de Telegram y la pregunta no tiene approver vivo
- Alternativa: aumentar el intervalo a 30s (el approver tiene retry de 3 min, 30s es suficiente)
- **Impacto:** -20 checks/min → 0 (o 2/min si se usa intervalo de 30s)

4.2 DESPERDICIO commander-launcher.js se ejecuta en CADA tool use

Situacion: Cada vez que Claude usa un tool (Read, Edit, Bash, Grep, etc.), `commander-launcher.js` lee el lockfile y verifica el PID. En una sesion tipica con ~200 tool uses, eso son 200 lecturas de lockfile + 200 `process.kill(pid, 0)`. El Commander casi nunca muere — el 99.5% de estas verificaciones son no-op.

Propuesta P-06: Agregar **cooldown de 60 segundos** al launcher, similar al health-check.

- Guardar timestamp de ultima verificacion en memoria (state file de 1 campo)
- Si pasaron menos de 60s desde la ultima verificacion exitosa, skip

- El Commander tiene heartbeat implicito (long-poll de 30s) — 60s de cooldown es seguro
- **Impacto:** -95% de reads de lockfile (de ~200 a ~10 por sesion)

4.3 DESPERDICIO health-check.js ejecuta gh CLI y Telegram getMe en cada check

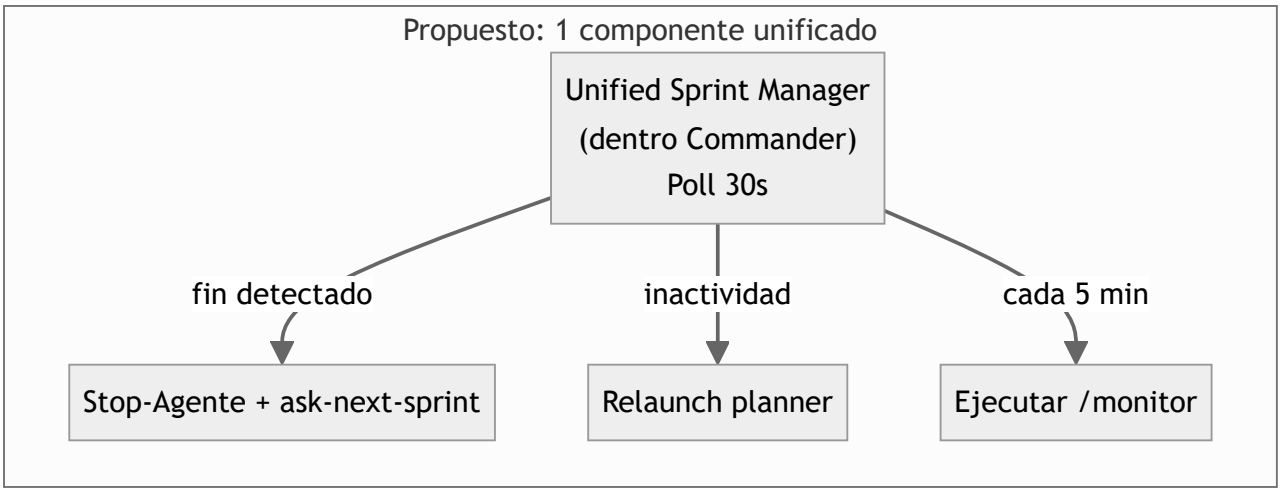
Situacion: Cada health check (cada 2-10 min) ejecuta `gh auth status` (spawn de proceso), `getMe` (HTTP a Telegram), y lee multiples lockfiles. La mayoria de estos checks son estables por horas.

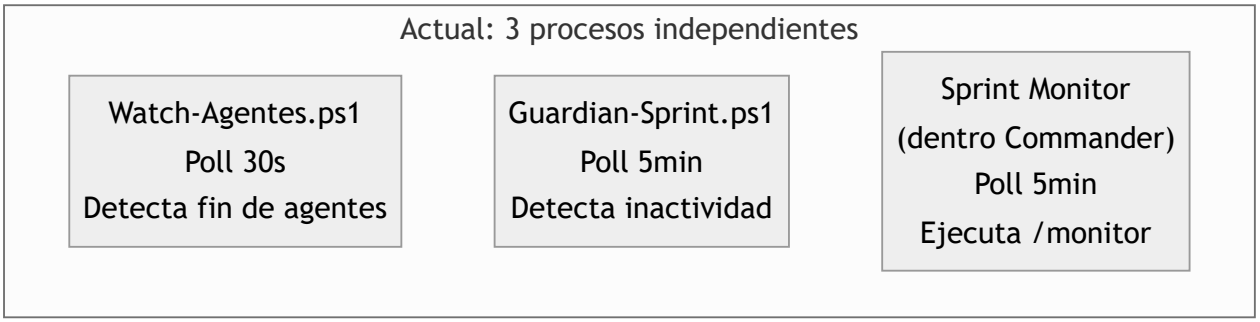
Propuesta P-07: Implementar **check selectivo con backoff exponencial** por componente:

- Si un check pasa 3 veces consecutivas, duplicar su intervalo (hasta MAX_INTERVAL_MS)
- Si falla, resetear a MIN_INTERVAL_MS solo para ESE check, no para todos
- Commander check: solo verificar PID (no HTTP) si el lockfile no cambio
- Telegram check: cachear resultado de `getMe` por 30 min (el bot no cambia)
- **Impacto:** -70% de llamadas HTTP y spawns de procesos

5. Oportunidades de consolidacion

5.1 ARQUITECTURA Unificar Watch-Agentes.ps1 + Guardian-Sprint.ps1 + Sprint Monitor del Commander





Propuesta P-08: Absorber la logica de Watch-Agentes y Guardian-Sprint **dentro del Commander**.

- El Commander ya tiene acceso a PIDs, worktrees, sprint-plan.json
- Ventajas: un solo proceso de polling, notificaciones centralizadas, estado compartido
- Watch-Agentes.ps1 ya usa `Test-AgentDone` basado en PIDs — migrar a Node.js es directo
- Guardian-Sprint.ps1 usa `Get-CimInstance Win32_Process` — equivalente Node.js: `execSync('wmic process...')` o `child_process`
- Mantener PS1 como fallback manual pero deprecar su uso automatico
- **Impacto:** -2 procesos PowerShell, -2 terminales abiertas, estado unificado

5.2 ARQUITECTURA Consolidar HTTP helpers duplicados

Problema: `telegramPost()` esta implementada identicamente en 4 archivos: `telegram-commander.js`, `permission-approver.js`, `ask-next-sprint.js`, `ci-monitor-bg.js`. Cada una tiene su propio manejo de timeout, error handling, y logging.

Propuesta P-09: Extraer a `telegram-client.js` compartido.

- Modulo: `const { sendMessage, editMessage, telegramPost } = require("./telegram-client")`
- Incluir: rate limiting, retry con backoff, logging centralizado
- Incluir: queue para P-02 (outbox pattern) como opcion
- **Impacto:** -200 lineas de codigo duplicado, comportamiento consistente

5.3 ARQUITECTURA Token de Telegram hardcodeado en Watch-Agentes.ps1

Problema de seguridad: `Watch-Agentes.ps1` linea 54 tiene el bot token hardcodeado: `$botToken = '8403197784:AAG07...'`. Esto es un secreto expuesto en un repo publico. Todos los demas scripts leen de `telegram-config.json`.

Propuesta P-10: Migrar a leer `telegram-config.json` como los demas.

- Reemplazar hardcoded con: `$cfg = Get-Content "$MainRepo\.claude\hooks\telegram-config.json" | ConvertFrom-Json`
- Rotar el token actual (ya esta expuesto en historial de git)
- **Impacto:** Elimina exposicion de secreto

6. Mejoras de performance

6.1 PERFORMANCE CI Monitor — Backoff progresivo

Propuesta P-11: El CI tipicamente tarda 3-8 minutos. Polling fijo de 30s desperdicia calls al inicio.

- Primeros 2 min: poll cada 60s (el workflow tarda en registrarse)
- Minutos 2-8: poll cada 30s (fase activa)
- Despues de 8 min: poll cada 60s (anormal, reducir carga)
- **Impacto:** -30% de API calls a GitHub Actions (~28 → ~18 calls promedio)

6.2 PERFORMANCE Permission Approver — Polling adaptativo

Propuesta P-12: El intervalo de 150ms es excesivo para la latencia humana (tipicamente >5s para responder).

- Primer segundo: 150ms (ultra-responsive para auto-approves)
- Segundos 1-10: 500ms (el humano esta leyendo)
- Despues de 10s: 1000ms (espera larga)

- Con P-01 (fs.watch): eliminar polling completamente
- **Impacto:** -80% de reads del archivo (de ~400/min a ~80/min en fase larga)

6.3 PERFORMANCE activity-logger.js — Batching de escrituras

Propuesta P-13: Cada tool use genera un `appendFileSync` al JSONL + un `writeFileSync` a sessions/. En rafagas de 10+ tools, esto son 20+ escrituras sincronas.

- Buffering: acumular en memoria por 2 segundos, luego flush batch
- Dado que es un hook y no un daemon, implementar via flag file: si el flag existe y tiene <2s, skip
- **Impacto:** -60% de I/O en rafagas

7. Propuesta de nuevo componente

7.1 NUEVO Process Supervisor — watchdog centralizado

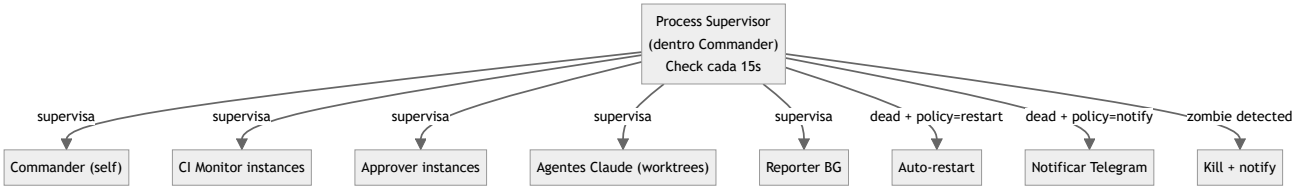
Justificacion: Actualmente la supervision de procesos esta fragmentada:

- `commander-launcher.js` supervisa al Commander (cada tool use)
- `health-check.js` supervisa Commander + approvers + bot + settings (cada 2-10 min)
- `Guardian-Sprint.ps1` supervisa agentes + worktrees (cada 5 min)
- `Watch-Agentes.ps1` supervisa terminales de agentes (cada 30s)

Son 4 supervisores con overlap significativo, cada uno con su propia logica de PID check.

Propuesta P-14: Crear `process-supervisor.js` como modulo singleton dentro del Commander.

- **Registry:** Hashtable de procesos registrados con PID, rol, last-heartbeat
- **Health loop:** Un solo `setInterval(15s)` que verifica todos los procesos
- **Auto-restart policy:** Configurable por proceso (restart, notify, ignore)
- **API:** `register(pid, role)`, `heartbeat(pid)`, `unregister(pid)`
- Los demas hooks solo llaman `supervisor.register()` al arrancar
- Reemplaza: commander-launcher (check), health-check (commander check), Guardian (claude check)



8. Bitacora de lecciones aprendidas — auto-actualizable

8.1 NUEVO ops-learnings.jsonl — Bitacora operativa auto-actualizable

Justificacion: Hoy las lecciones aprendidas se pierden entre logs de debug (`hook-debug.log`), comentarios en codigo, y memoria del desarrollador. No hay un sistema que **capture automaticamente** patrones de error, los clasifique, y los ponga disponibles para que los agentes aprendan de errores pasados antes de repetirlos.

Ejemplos de lecciones que se descubrieron "a la fuerza" y se podrian haber evitado:

- Junctions NTFS + `rm -rf` borran el `.claude/` real (incidente 2026-02-27)
- `-gpu host` no funciona en headless + Intel Iris Xe + WHPX
- `ask-next-sprint.js` causa 409 si Commander esta corriendo
- El polling a 150ms del approver satura I/O en sesiones largas

Propuesta P-15: Crear sistema de bitacora auto-actualizable con dos componentes:

Componente A: Captura automatica (hooks)

- **Archivo:** `.claude/hooks/ops-learnings.jsonl` (append-only, max 500 entries)
- **Formato por entrada:**

```
{
  "ts": "2026-03-04T15:30:00Z",
  "source": "health-check",          // hook que detecto el problema
  "category": "contention|crash|perf|config|security",
  "severity": "critical|high|medium|low",
  "symptom": "409 Conflict en getUpdates",
  "root_cause": "Dos procesos compiten por Telegram long-poll",
  "resolution": "Migrar ask-next-sprint a pending-questions",
  "affected": ["ask-next-sprint.js", "telegram-commander.js"],
  "auto_detected": true,              // vs manual
  "times_seen": 12,                   // contador incremental
  "first_seen": "2026-02-15T...",
  "last_seen": "2026-03-04T...",
```

```
  "status": "open|mitigated|resolved"
}
```

Componente B: Deteccion y registro automatico

Cada hook que detecta un error imprevisto o comportamiento anomalo lo registra en la bitacora:

Hook	Que detecta automaticamente
`health-check.js`	Commander caido, approvers huerfanos, bot no responde, settings corruptos
`ci-monitor-bg.js`	CI timeout, CI fallo recurrente en misma rama, GitHub API rate limited
`permission-approver.js`	Timeout de aprobacion (usuario no responde), PQ file corrupto
`commander-launcher.js`	Relanzamientos frecuentes (>3/hora = inestabilidad), race conditions
`telegram-commander.js`	409 conflicts, exec timeout, sprint stalled, zombie approvers
`Guardian-Sprint.ps1`	Zombie agents (CPU=0), relanzamientos frecuentes
`activity-logger.js`	Sesiones sin actividad por >15 min (agente bloqueado silenciosamente)

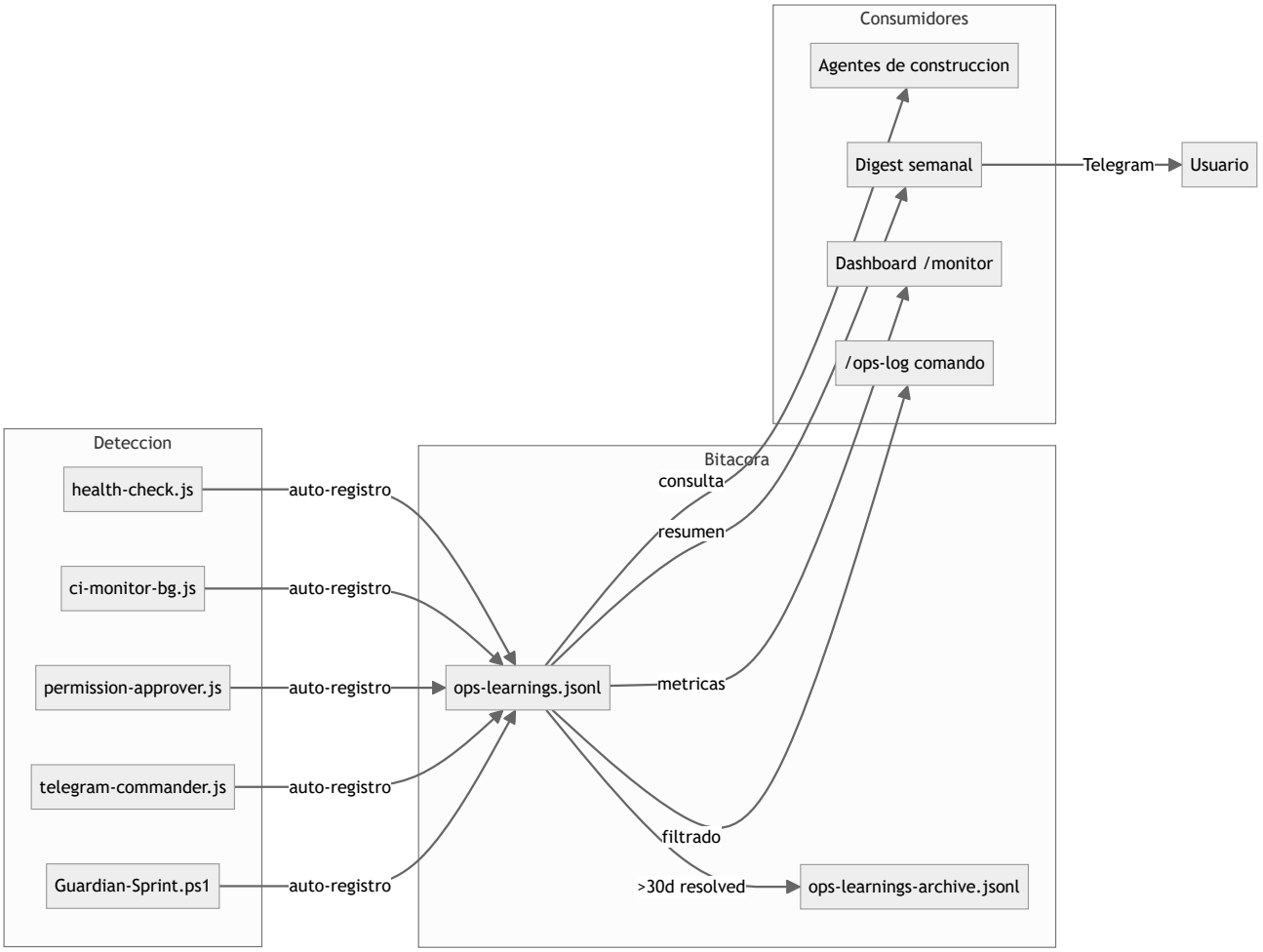
Componente C: Consulta y aprendizaje

- **Comando:** `/ops-log` o endpoint en Commander: lista las ultimas N lecciones, filtra por categoria/severity
- **Integracion con agentes:** Antes de implementar cambios en hooks, el agente consulta la bitacora para verificar si hay lecciones relevantes
- **Digest semanal:** El Commander genera un resumen semanal de lecciones nuevas y lo envia a Telegram
- **Auto-prune:** Entradas con `status: "resolved"` y `last_seen` > 30 dias se archivan a `ops-learnings-archive.jsonl`

Componente D: Auto-actualizacion proactiva

- Cuando un hook detecta un error por **segunda vez**, incrementa `times_seen` y escala la severidad si supera umbrales (3x → high, 5x → critical)
- Cuando un problema previamente registrado **no se ve en 7 dias**, se marca como `mitigated`
- Cuando se implementa un fix (detectado via commit message pattern `fix(hooks):`), se busca en la bitacora y se marca como `resolved` automaticamente

- Los agentes de construccion (`/backend-dev` , etc.) consultan la bitacora al inicio de su sesion para evitar repetir errores conocidos



8.2 NUEVO Integracion con MEMORY.md — Lecciones persistentes cross-sesion

Propuesta P-16: Cuando una leccion alcanza severidad `critical` o `times_seen >= 5` , el sistema la promueve automaticamente a `MEMORY.md` o un archivo de memoria dedicado (`ops-lessons.md`).

- Formato en memoria: una linea por leccion con sintaxis compacta
- Ejemplo: `- **NUNCA** usar getUpdates desde scripts externos si Commander esta activo (causa 409) [2026-02-15, resuelto 2026-03-04]`
- Esto asegura que Claude Code (en cualquier sesion futura) tenga acceso a las lecciones mas criticas sin necesidad de leer el JSONL completo
- Auto-cleanup: cuando una leccion se marca como `resolved` por >30 dias, se puede mover a un archivo de archivo

9. Resumen de propuestas priorizadas

ID	Propuesta	Severidad	Esfuerzo	Impacto
P-10	Eliminar token hardcodeado en Watch-Agentes.ps1	**Critico**	XS (5 min)	Seguridad
P-01	IPC/fs.watch para pending-questions.json	**Critico**	M (2-3h)	Elimina corrupcion + -400 reads/min
P-03	ask-next-sprint via PQ (sin getUpdates)	**Alto**	S (1h)	Elimina 409 conflicts
P-04	Lock atomico con fs.openSync('wx')	**Alto**	XS (15 min)	Elimina race condition
P-05	Eliminar orphan detection interval (3s)	Medio	XS (10 min)	-20 checks/min
P-06	Cooldown 60s en commander-launcher	Medio	XS (15 min)	-95% reads de lockfile
P-12	Polling adaptativo en permission-approver	Medio	S (30 min)	-80% reads de PQ file
P-07	Health check selectivo con backoff por componente	Medio	M (2h)	-70% HTTP calls + spawns
P-11	CI Monitor con backoff progresivo	Medio	S (30 min)	-30% GitHub API calls
P-09	Extraer telegram-client.js compartido	Bajo	M (2h)	-200 lineas duplicadas
P-02	Message broker centralizado en Commander	Bajo	L (4h)	Elimina 429 potential
P-08	Unificar Watch+Guardian+Sprint Monitor	Bajo	L (6h)	-2 procesos, estado unificado
P-13	Batching en activity-logger	Bajo	S (30 min)	-60% I/O en rafagas
P-14	Process Supervisor centralizado	Bajo	XL (8h)	Simplifica toda la supervision
P-15	Bitacora ops-learnings.jsonl auto-actualizable	**Alto**	M (3h)	Captura automatica de lecciones, previene errores recurrentes
P-16	Promocion automatica de lecciones criticas a MEMORY.md	Medio	S (1h)	Lecciones persistentes cross-sesion

9. Plan de implementacion sugerido

Fase 1 — Quick wins (1 dia)

- **P-10** Eliminar token hardcodeado + rotar token
- **P-04** Lock atomico con `fs.openSync('wx')`
- **P-05** Eliminar/reducir orphan detection interval
- **P-06** Cooldown 60s en commander-launcher

Fase 2 — Contención de recursos (2-3 dias)

- **P-01** Reemplazar polling PQ con fs.watch + escritura atomica
- **P-03** Migrar ask-next-sprint a PQ-only
- **P-12** Polling adaptativo en permission-approver
- **P-09** Extraer telegram-client.js

Fase 3 — Optimizacion (1-2 dias)

- **P-07** Health check selectivo con backoff
- **P-11** CI Monitor con backoff progresivo
- **P-13** Batching en activity-logger
- **P-02** Message broker en Commander

Fase 4 — Observabilidad y mejora continua (1-2 dias)

- **P-15** Bitacora ops-learnings.jsonl auto-actualizable
- **P-16** Promocion automatica de lecciones criticas a MEMORY.md

Fase 5 — Consolidacion arquitectonica (3-4 dias)

- **P-08** Unificar Watch+Guardian+Sprint Monitor en Commander
- **P-14** Process Supervisor centralizado

10. Metricas de exito esperadas

Metrica	Actual	Esperado post-mejoras	Reduccion
Reads de pending-questions.json	~400/min	~0 (event-driven)	~100%
Reads de lockfile (Commander)	~200/sesion	~10/sesion	~95%
Orphan checks	20/min	0-2/min	~90%
Telegram API calls en idle	~8/min	~3/min	~62%
GitHub API calls (CI)	~28/run	~18/run	~36%

Procesos concurrentes (sprint)	5-6	3-4	~33%
Lineas de codigo duplicado	~350	~100	~71%
Riesgo de corrupcion PQ	Alto	Nulo	Eliminado

Generado automaticamente por Claude Opus 4.6 — Intrale Platform — 2026-03-04